

DS ASSIGNMENT SET-4

-Cristina Jane B

1.Implement a Stack using Queue data structure

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int queue[MAX];
```

```
int front = 0, rear = -1;
```

```
void push(int value)
```

```
{
```

```
    int i, temp;
```

```
    if (rear == MAX - 1)
```

```
    {
```

```
        printf("Stack Overflow\n");
```

```
        return;
```

```
    }
```

```
    queue[++rear] = value;
```

```
    for (i = front; i < rear; i++)
```

```
    {
```

```
        temp = queue[i];
```

```
        for (int j = i; j < rear; j++)
```

```
            queue[j] = queue[j + 1];
```

```
    queue[rear] = temp;  
}
```

```
    printf("%d pushed into stack\n", value);  
}
```

```
void pop()  
{  
    if (front > rear)  
    {  
        printf("Stack Underflow\n");  
        return;  
    }  
}
```

```
    printf("%d popped from stack\n", queue[front++]);  
}
```

```
void display()  
{  
    if (front > rear)  
    {  
        printf("Stack is empty\n");  
        return;  
    }  
}
```

```
printf("Stack elements: ");
```

```
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);

    printf("\n");
}

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n--- Stack using Queue ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                push(value);
                break;
```

case 2:

pop();

break;

case 3:

display();

break;

case 4:

return 0;

default:

printf("Invalid choice\n");

}

}

}

OUTPUT:

```

--- Stack using Queue ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value: 41
41 pushed into stack

--- Stack using Queue ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements: 41

--- Stack using Queue ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
PS C:\Users\B.Cristina Jane\OneDrive

```

2.Implement a Queue using stack data structure

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack1[MAX], stack2[MAX];
```

```
int top1 = -1, top2 = -1;
```

```
void push1(int value)
```

```
{
```

```
    if (top1 == MAX - 1)
```

```
        printf("Queue Overflow\n");
```

```
    else
```

```
        stack1[++top1] = value;
```

```
}
```

```
void push2(int value)
```

```
{
```

```
    stack2[++top2] = value;
```

```
}
```

```
int pop1()
```

```
{
```

```
    return stack1[top1--];
```

```
}
```

```
int pop2()
```

```
{
```

```
    return stack2[top2--];
```

```
}
```

```
void enqueue(int value)
```

```
{
```

```
    if (top1 == MAX - 1)
```

```
    {
```

```
        printf("Queue Overflow\n");
```

```
        return;
```

```
    }
```

```
    push1(value);
```

```
    printf("%d inserted into queue\n", value);
```

```
}
```

```
void dequeue()
```

```
{
    int value;

    if (top1 == -1 && top2 == -1)
    {
        printf("Queue Underflow\n");
        return;
    }
    if (top2 == -1)
    {
        while (top1 != -1)
        {
            value = pop1();
            push2(value);
        }
    }

    // Remove the front element
    value = pop2();

    printf("%d deleted from queue\n", value);
}

void display()
{
    int i;

    if (top1 == -1 && top2 == -1)
```

```

{
    printf("Queue is empty\n");
    return;
}

printf("Queue elements: ");

for (i = top2; i >= 0; i--)
    printf("%d ", stack2[i]);

// Elements in stack1
for (i = 0; i <= top1; i++)
    printf("%d ", stack1[i]);

printf("\n");
}

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n--- Queue using Stack ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
    }
}

```



```
printf("4. Exit\n");
```

```
printf("Enter your choice: ");
```

```
scanf("%d", &choice);
```

```
switch (choice)
```

```
{
```

```
    case 1:
```

```
        printf("Enter value: ");
```

```
        scanf("%d", &value);
```

```
        enqueue(value);
```

```
        break;
```

```
    case 2:
```

```
        dequeue();
```

```
        break;
```

```
    case 3:
```

```
        display();
```

```
        break;
```

```
    case 4:
```

```
        return 0;
```

```
    default:
```

```
        printf("Invalid choice\n");
```

```
}
```

```
}
```

```
}
```

OUTPUT:

```
--- Queue using Stack ---
```

```
1. Enqueue
```

```
2. Dequeue
```

```
3. Display
```

```
4. Exit
```

```
Enter your choice: 1
```

```
Enter value: 33
```

```
33 inserted into queue
```

```
--- Queue using Stack ---
```

```
1. Enqueue
```

```
2. Dequeue
```

```
3. Display
```

```
4. Exit
```

```
Enter your choice: 1
```

```
Enter value: 56
```

```
56 inserted into queue
```

```
--- Queue using Stack ---
```

```
1. Enqueue
```

```
2. Dequeue
```

```
3. Display
```

```
4. Exit
```

```
Enter your choice: 2
```

```
33 deleted from queue
```

```
--- Queue using Stack ---
```

```
1. Enqueue
```

```
2. Dequeue
```

```
3. Display
```

```
4. Exit
```

```
Enter your choice: 3
```

```
Queue elements: 56
```

```
--- Queue using Stack ---
```

```
1. Enqueue
```

```
2. Dequeue
```

```
3. Display
```

```
4. Exit
```

```
Enter your choice: 4
```

```
D:\C++\Heaps\B_Cristina_7
```

3. Write a C program using stack data structure to implement managing the history of web pages visited.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 10
```

```
char history[MAX][100];
```

```
int top = -1;
```

```
void visitPage()
```

```
{
```

```
    char page[100];
```

```
    if (top == MAX - 1)
```

```
    {
```

```
        printf("History is full!\n");
```

```
        return;
```

```
    }
```

```
    printf("Enter webpage URL: ");
```

```
    scanf("%s", page);
```

```
    top++;
```

```
    strcpy(history[top], page);
```

```
    printf("Visited: %s\n", page);
```

```
}
```

```
void goBack()
```

```
{
```

```
    if (top == -1)
```

```
    {
```

```
        printf("No previous page available.\n");
```

```
        return;
```

```
    }
```

```
    printf("Going back from: %s\n", history[top]);
```

```
    top--;
```

```
    if (top == -1)
```

```
        printf("No webpage in history.\n");
```

```
    else
```

```
        printf("Current page: %s\n", history[top]);
```

```
}
```

```
void displayHistory()
```

```
{
```

```
    int i;
```

```
    if (top == -1)
```

```
    {
```

```
        printf("History is empty.\n");
```

```
        return;
```

```
    }
```

```
printf("\nWeb Page History:\n");
```

```
for (i = top; i >= 0; i--)
```

```
{
```

```
    printf("%s\n", history[i]);
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while (1)
```

```
{
```

```
    printf("\n--- Browser History ---\n");
```

```
    printf("1. Visit Web Page\n");
```

```
    printf("2. Go Back\n");
```

```
    printf("3. Display History\n");
```

```
    printf("4. Exit\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch (choice)
```

```
{
```

```
        case 1:
```

```
            visitPage();
```

```
            break;
```

case 2:

goBack();

break;

case 3:

displayHistory();

break;

case 4:

return 0;

default:

printf("Invalid choice!\n");

}

}

}

OUTPUT:

```
--- Browser History ---
1. Visit Web Page
2. Go Back
3. Display History
4. Exit
Enter your choice: 1
Enter webpage URL: github.com
Visited: github.com
```

```
--- Browser History ---
1. Visit Web Page
2. Go Back
3. Display History
4. Exit
Enter your choice: 1
Enter webpage URL: google.com
Visited: google.com
```

```
--- Browser History ---
1. Visit Web Page
2. Go Back
3. Display History
4. Exit
Enter your choice: 2
Going back from: google.com
Current page: github.com
```

```
--- Browser History ---
1. Visit Web Page
2. Go Back
3. Display History
4. Exit
Enter your choice: 3
```

```
Web Page History:
github.com
```

```
--- Browser History ---
1. Visit Web Page
2. Go Back
3. Display History
4. Exit
Enter your choice: 4
```

4. Write a C program to sort a stack using temporary stack

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int stack[MAX], top = -1;
```

```
int temp[MAX], ttop = -1;
```

```
void push(int stack[], int *top, int value)
```

```
{  
    stack[++(*top)] = value;  
}
```

```
int pop(int stack[], int *top)
```

```
{  
    return stack[(*top)--];  
}
```

```
void sortStack()
```

```
{  
    while (top != -1)  
    {  
        int x = pop(stack, &top);  
  
        while (ttop != -1 && temp[ttop] > x)  
        {  
            push(stack, &top, pop(temp, &ttop));  
        }  
    }  
}
```



```
    push(temp, &ttop, x);  
}
```

```
while (ttop != -1)  
{  
    push(stack, &top, pop(temp, &ttop));  
}  
}
```

```
void display()  
{  
    printf("Sorted Stack: ");  
  
    for (int i = top; i >= 0; i--)  
    {  
        printf("%d ", stack[i]);  
    }  
}
```

```
int main()  
{  
    int n, value;  
  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    printf("Enter stack elements:\n");
```

```

for (int i = 0; i < n; i++)
{
    scanf("%d", &value);
    push(stack, &top, value);
}

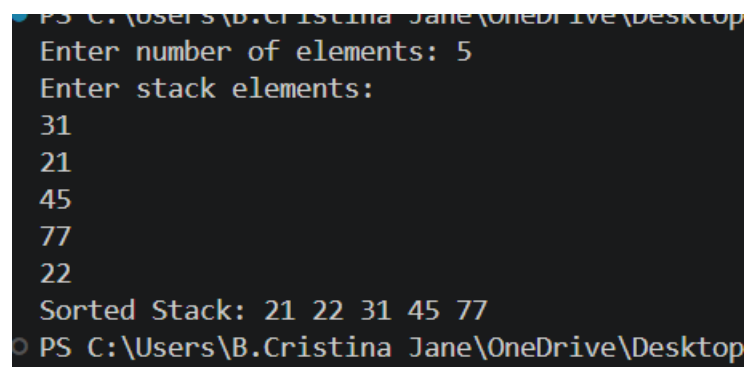
sortStack();

display();

return 0;
}

```

OUTPUT:



```

PS C:\Users\B.Cristina Jane\OneDrive\Desktop
Enter number of elements: 5
Enter stack elements:
31
21
45
77
22
Sorted Stack: 21 22 31 45 77
PS C:\Users\B.Cristina Jane\OneDrive\Desktop

```

The stock span problem is a financial problem where we have a series of daily price Quotes for a stock denoted by an array and the task is to calculate the span of the stocks price for all days . The span of a stock's price for ith day gives the maximum number of consecutive days leading up to ith day (including current day) where stocks price is less than or equal to its price on day i.e

eg input:[100,80,60,70,60,75,85] and

output:[1,1,1,2,1,4,6]

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main()
{
    int price[MAX], span[MAX];
    int stack[MAX];
    int top = -1;
    int n;

    printf("Enter number of days: ");
    scanf("%d", &n);

    printf("Enter stock prices:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &price[i]);
    }
    span[0] = 1;
    stack[++top] = 0;

    for (int i = 1; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
        {
            top--;
        }

        if (top == -1)
```

```
{
    span[i] = i + 1;
}
else
{
    span[i] = i - stack[top];
}

    stack[++top] = i;
}

printf("Stock Prices: ");
for (int i = 0; i < n; i++)
{
    printf("%d ", price[i]);
}

printf("\nStock Span: ");
for (int i = 0; i < n; i++)
{
    printf("%d ", span[i]);
}

return 0;
}
```

OUTPUT:

```
Enter number of days: 7
Enter stock prices:
100 80 60 70 60 75 85
Stock Prices: 100 80 60 70 60 75 85
Stock Span:   1 1 1 2 1 4 6
PS C:\Users\B. Cristina Lane\OneDrive\Desktop\
```

6. Given an array for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists set value to -1. Implement using stack.

I/P: [2, 1, 1, 3, 2, 1] and

O/P: [1, -1, -1, 2, 1, -1]

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main()
```

```
{
```

```
    int arr[MAX], freq[MAX], stack[MAX];
```

```
    int result[MAX];
```

```
    int n, top = -1;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    for (int i = 0; i < n; i++)
```

```

{
    freq[i] = 0;

    for (int j = 0; j < n; j++)
    {
        if (arr[i] == arr[j])
            freq[i]++;
    }
}

for (int i = n - 1; i >= 0; i--)
{
    while (top != -1 && freq[stack[top]] <= freq[i])
    {
        top--;
    }

    if (top == -1)
        result[i] = -1;
    else
        result[i] = arr[stack[top]];

    stack[++top] = i;
}

printf("Output: ");
for (int i = 0; i < n; i++)
{
    printf("%d ", result[i]);
}

```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:

```
Enter number of elements: 6
Enter elements:
2 1 1 3 2 1
Output: 1 -1 -1 2 1 -1
```

7. Given a pattern containing only I's and D's. I for increasing and 'D' for decreasing. Device an algorithm using stack to print minimum number following that pattern .Digits from 1-9 and digits can't repeat

EX1:

I/P:D

O/P:21

EX2:

I/P:DIDI

O/P 21435

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main()
```

```
{
```

```
    char pattern[MAX];
```

```
    int stack[MAX];
```

```
    int top = -1;
```

```
    int num = 1;
```

```
printf("Enter pattern (I/D): ");
scanf("%s", pattern);

printf("Minimum number: ");

for (int i = 0; pattern[i] != '\0'; i++)
{
    // Push the next number
    stack[++top] = num++;

    // If I, print everything in stack
    if (pattern[i] == 'I')
    {
        while (top != -1)
        {
            printf("%d", stack[top--]);
        }
    }
}

// Push the last number
stack[++top] = num++;

// Print remaining elements
while (top != -1)
{
    printf("%d", stack[top--]);
}
```



```
    return 0;
}
```

OUTPUT:

```
PS C:\Users\B.Cristina Jane\OneDrive\Desktop>
Enter pattern (I/D): D
Minimum number: 21
PS C:\Users\B.Cristina Jane\OneDrive\Desktop>
```

8. Given an array of digits (values are from 0 to 9) find the minimum possible sum of two numbers formed from digits of the array. All digits of given array using we used to form the two numbers we used to form the two numbers. Use Queue for implementation.

I/P: [6, 8, 4, 5, 2, 3]

O/P: 604

(min sum formed by no's 358 and 246)

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int queue[MAX];
```

```
int front = 0, rear = -1;
```

```
void enqueue(int x)
```

```
{
```

```
    queue[++rear] = x;
```

```
}
```

```
int dequeue()
```

```
{
```

```
    return queue[front++];
```

```
}
```

```
int main()
```

```
{
```

```
    int arr[MAX];
```

```
    int n;
```

```
    int num1 = 0, num2 = 0;
```

```
    printf("Enter number of digits: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the digits:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    for (int i = 0; i < n - 1; i++)
```

```
    {
```

```
        for (int j = i + 1; j < n; j++)
```

```
        {
```

```
            if (arr[i] > arr[j])
```

```
            {
```

```
                int temp = arr[i];
```

```
                arr[i] = arr[j];
```

```
                arr[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
for (int i = 0; i < n; i++)
{
    enqueue(arr[i]);
}

int turn = 0;

while (front <= rear)
{
    int digit = dequeue();

    if (turn == 0)
    {
        num1 = num1 * 10 + digit;
        turn = 1;
    }
    else
    {
        num2 = num2 * 10 + digit;
        turn = 0;
    }
}

printf("Number 1: %d\n", num1);
printf("Number 2: %d\n", num2);
printf("Minimum Sum: %d\n", num1 + num2);

return 0;
}
```

OUTPUT:

```
PS C:\Users\B.Cristina Jane\OneDri
Enter number of digits: 6
Enter the digits:
6 8 4 5 2 3
Number 1: 246
Number 2: 358
Minimum Sum: 604
PS C:\Users\B.Cristina Jane\OneDri
```

9. Given an array and a positive integer K, find the first negative integer for each window (consecutive subarray) of size K. If a window does not contain a negative integer, then print 0 for that window.

Implement using Queue.

Input: [-8, 2, 3, -6, 1], K = 2

Output: [-8, 0, -6, -6]

first negative integer for each window of size 2

[-8,2] = -8

[2,3] = 0 (No negative)

[3,-6] = -6

[-6,1] = -6

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main()
```

```
{
```

```
    int arr[MAX], queue[MAX];
```

```
    int front = 0, rear = -1;
```

```
    int n, k;
```

```
    printf("Enter number of elements: ");
```

```
scanf("%d", &n);
```

```
printf("Enter the elements:\n");
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    scanf("%d", &arr[i]);
```

```
}
```

```
printf("Enter window size K: ");
```

```
scanf("%d", &k);
```

```
printf("Output: ");
```

```
for (int i = 0; i <= n - k; i++)
```

```
{
```

```
    front = 0;
```

```
    rear = -1;
```

```
    for (int j = i; j < i + k; j++)
```

```
    {
```

```
        if (arr[j] < 0)
```

```
        {
```

```
            queue[++rear] = arr[j];
```

```
        }
```

```
    }
```

```
    if (front <= rear)
```

```
        printf("%d ", queue[front]);
```

```
    else
```

```
        printf("0 ");
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:

```
Enter number of elements: 5
Enter the elements:
-8 2 3 -6 1
Enter window size K: 2
Output: -8 0 -6 -6
```

10. Given a matrix of dimension $m \times n$ where each cell in the matrix can have values 0, 1 or 2 which has the following meaning : 0 : empty cell 1 : cells having fresh oranges 2 : cells having rotten oranges The task is to find the minimum time required so that all oranges become rotten . A rotten orange at index (i,j) can rot either fresh orange which are its neighbours (up, down, left and right). If impossible to rot every orange then return -1. Use Queue for implementation .

Example

I/P: [2, 1, 0, 2, 1] [1, 0, 1, 2, 1] [1, 0, 0, 2, 1]

O/P: 2

explanation :

At 0th frame

[2 1 0 2 1

1 0 1 2 1

1 0 0 2 1]

At 1st frame

[2 2 0 2 2

2 0 2 2 2

1 0 0 2 2]

At 2nd frame

[2 2 0 2 2

```
2 0 2 2 2
```

```
2 0 0 2 2]
```

```
#include <stdio.h>
```

```
#define MAX 100
```

```
struct Node
```

```
{
```

```
    int row;
```

```
    int col;
```

```
};
```

```
int main()
```

```
{
```

```
    int matrix[MAX][MAX];
```

```
    struct Node queue[MAX * MAX];
```

```
    int front = 0, rear = -1;
```

```
    int m, n;
```

```
    int fresh = 0;
```

```
    int time = 0;
```

```
    printf("Enter rows and columns: ");
```

```
    scanf("%d %d", &m, &n);
```

```
    printf("Enter matrix elements:\n");
```

```
    for (int i = 0; i < m; i++)
```

```
{
```

```

for (int j = 0; j < n; j++)
{
    scanf("%d", &matrix[i][j]);

    if (matrix[i][j] == 2)
    {
        queue[++rear].row = i;
        queue[rear].col = j;
    }
    else if (matrix[i][j] == 1)
    {
        fresh++;
    }
}

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1}
while (front <= rear && fresh > 0)
{
    int size = rear - front + 1;
    for (int i = 0; i < size; i++)
    {
        int r = queue[front].row;
        int c = queue[front].col;
        front++;

        for (int d = 0; d < 4; d++)
        {

```



```

int nr = r + dr[d];
int nc = c + dc[d];

if (nr >= 0 && nr < m &&
    nc >= 0 && nc < n &&
    matrix[nr][nc] == 1)
{
    matrix[nr][nc] = 2;
    fresh--;

    queue[++rear].row = nr;
    queue[rear].col = nc;
}
}
}

time++;
}

if (fresh > 0)
    printf("Minimum time: -1\n");
else
    printf("Minimum time: %d\n", time);

return 0;
}

```

OUTPUT:

Enter rows and columns: 3 5

Enter matrix elements:

2 1 0 2 1

1 0 1 2 1

1 0 0 2 1

Minimum time: 2